NEO4J И CYPHER: ГРАФ УНИВЕРСИТЕТСКОГО КАМПУСА

Модель корпусов, дорог и маршрутов в графовой базе данных.

,01.

ЗАДАЧА ПРОЕКТА — НАХОДИТЬ МАРШРУТЫ ВНУТРИ КАМПУСА

1. Представить учебные корпуса и дороги как граф.

2. **Искать оптимальные маршруты** по времени и числу переходов.

3. Показать CRUD, фильтрацию и анализ связности в Neo4j.

Проект развёрнут в облачной среде Neo4j AuraDB.

02 · МОДЕЛЬ МЫШЛЕНИЯ

ПОЧЕМУ ЗДЕСЬ ПОДХОДИТ ГРАФОВАЯ БАЗА ДАННЫХ

УЗЛЫ

Корпуса—узлы,дороги — связи между узлами.

ПУТЬ

Последовательность

Маршрутчитается как последовательность связанных объектов.

CYPHER

Cypher описывает, что найти, без ручной реализации BFS или DFS.

Модель рассчитана на навигацию и анализ топологии кампуса.

03 · СХЕМА ДАННЫХ

СХЕМА ДАННЫХ: BUILDING И CONNECTED_TO

Building

id, name, floors.

CONNECTED_TO

distance_m, time_min.

Связи направленные; для движения в обе стороны создаются две противоположные связи.

Получается взвешенный связный граф.

ПОДГОТОВКА БАЗЫ И СОЗДАНИЕ ИСХОДНЫХ ДАННЫХ

ОЧИСТКА

MATCH (n)
DETACH DELETE n
Удаляет узлы вместе со связями.

СОЗДАНИЕ

CREATE
пять Building B1–B5
Формирует корпуса с id, названием и этажностью.

СВЯЗИ

MATCH два корпуса
CREATE CONNECTED_TO
Добавляет дорогу с расстоянием и временем.

Сначала очищаем граф, затем наполняем его данными.

05 · ЗАПРОС 1

ВИЗУАЛИЗИРОВАТЬ ПОЛНЫЙ ГРАФ

```
MATCH (n)-[r:CONNECTED_TO]->
(m)
RETURN n, r, m;
```

MATCH: находит пары узлов и связи между узлами.

r:CONNECTED_TO: ограничивает поиск дорогами нужного типа.

RETURN: возвращает узлы и связи для просмотра в Neo4j Browser.

Роль запроса — быстро проверить, что топология построена корректно.

06 · ЗАПРОС 2

ЗАПРОС 2 — ВЫВЕСТИ КОРПУСА ПО ЭТАЖНОСТИ

```
MATCH (b:Building)
RETURN b.id AS ID, b.name AS Название, b.floors AS Этажность
ORDER BY b.floors DESC;
```

- **b:Building:** выбирает все узлы-корпуса.
- **AS:** задаёт понятные имена столбцов результата.
- **ORDER BY … DESC:** сортирует от большей этажности к меньшей.

Роль запроса — обычное чтение и сортировка свойств узлов.

07 · ЗАПРОС 3

НАЙТИ СОСЕДЕЙ ГЛАВНОГО КОРПУСА

```
MATCH (b:Building {id: 'B1'})-
[r:CONNECTED_TO]->(target:Building)
RETURN target.name AS Соседний_Корпус,
       r.distance_m AS Расстояние_м,
       r.time_min AS Время_мин;
```



ЗАПРОС 4 — НАЙТИ КРАТЧАЙШИЙ ПУТЬ ПО ЧИСЛУ ПЕРЕХОДОВ

```
MATCH path = shortestPath((s:Building {id: 'B1'})-
[:CONNECTED_TO*]-(e:Building {id: 'B5'}))
RETURN [node in nodes(path) | node.name] AS Маршрут,
reduce(total_time = 0, r in relationships(path) | total_time +
r.time_min) AS Общее_время_мин;
```

1. **shortestPath:** ищет путь с минимальным числом рёбер.
2. **nodes(path):** получает узлы маршру та.
3. **reduce:** суммирует время всех связей пу ти.

Важно: кратчайший по переходам маршрут не всегда минимален по времени.

09 · ЗАПРОС 5

ПРОВЕРИТЬ АЛЬТЕРНАТИВНЫЕ МАРШРУТЫ

```
MATCH path = (s:Building {id: 'B1'})-[:CONNECTED_TO*]-(e:Building {id: 'B5'})
WITH path, reduce(t = 0, r in relationships(path) |
 t + r.time_min) AS total_time
WHERE total_time <= 10
RETURN [n in nodes(path) | n.name] AS Вариант_маршрута,
       total_time AS Время_в_пути;
```

WITH: вычисляет промежуточное значение total_time. **WHERE:** оставляет только маршруты до 10 минут. **RETURN:** показывает все подходящие варианты.

Роль запроса — фильтрация путей по реальному весу времени.

NEO4J · CYPHER · 06—08

Neo4j и Cypher: запросы 6–8

Каканализировать связи, обновлять свойства и удалять отношения в графовой базе данных.

Граф — это данные и отношения

Building	узлы графа
CONNECTED_TO	дороги между ними
Cypher	поиск и изменение

Три запроса: увидеть структуру, изменить свойство, удалить отношение.

ПРОВЕРЯЕМЫЕ ОПЕРАЦИИ

Запрос 6

Показывает степень узлов: сколько дорог связано с каждым корпусом.

Запрос 7

Изменяет время пути между B1 и B2.

Запрос 8

Удаляет дорогу между B4 и B5 при реконструкции.

ЗАПРОС 06 · АНАЛИЗ СТЕПЕНИ УЗЛА

Запрос 6 находит самые связанные корпуса

Cypher

```
MATCH (b:Building)
RETURN b,
       size([(b)-[:CONNECTED_TO]-() |
1])
        AS Количество_дорог
ORDER BY Количество_дорог DESC
```

Возвращает корпус и количество дорог; сортирует от большего к меньшему.

MATCH: находит все узлы с меткой Building.

size([...]): считает найденные соседние связи.

ORDER BY DESC: ставит наиболее связанные корпуса первыми.

ЗАПРОС 06 · ПОСТРОЧНЫЙ РАЗБОР

Как читать код запроса 6

`(b:Building)`	**Переменная b:** обозначает найденный корпус.
`(b)-[:CONNECTED_TO]-()`	**Любое направление:** ищет дороги в любом направлении.
`| 1`	**Единица на связь:** превращает каждую найденную связь в единицу для подсчёта.
`AS Количество_дорог`	**Имя результата:** задаёт понятное имя столбцу результата.

ЗАПРОС 07 · UPDATE СВОЙСТВА

Запрос 7 выполняет UPDATE свойства связи

Cypher

```
MATCH (b1:Building {name: "B1"})
-[r:CONNECTED_TO]-
(b2:Building {name: "B2"})
SET r.time_min = 2.0
RETURN b1, r, b2
```

Время пути между Главным корпусом B1 и корпусом B2 — 2 минуты.

БЫЛО: TIME_MIN → SET: 2.0

MATCH: находит два корпуса и связь CONNECTED_TO.
SET r.time_min = 2.0: записывает новое значение свойства.
RETURN: сразу показывает направление и обновлённое время.

ЗАПРОС 08 · DELETE ОТНОШЕНИЯ

Запрос 8 удаляет временно закрытую дорогу

Cypher

```
MATCH (b4:Building {name: "B4"})
    -[r:CONNECTED_TO]-
    (b5:Building {name: "B5"})
DELETE r
```

01
B4
Библиотека

02
DELETE r
удаляется
связь

03
B5
Спортивны
й комплекс

Связь между Библиотекой B4 и Спортивным комплексом B5 удаляется как модель реконструкции.

Узлы B4 и B5 остаются. После удаления маршрут через эту дорогу больше не используется.

MATCH
находит нужные корпуса и их связь.

DELETE r
удаляет именно отношение r.

Узлы остаются
удаляется дорога, не корпуса.

ИТОГ · 06—08

Итог: три операции над графом

Запрос	Операция	Ключевое слово	Практический результат
Запрос 6	анализ	MATCH	Определяет узловые корпуса по числу дорог.
Запрос 7	обновление	UPDATE	Меняет свойство существующей связи.
Запрос 8	удаление	DELETE	Удаляет связь, сохраняя корпуса.

Cypher описывает, что найти и изменить, без ручного обхода графа.